

Converting between Sequent Systems and Truth Trees

Andrew Nazareth and Andrii Bonenberger

April 26, 2026

Sequent System Rules (Propositional Calculus)

The sequent system we are using has the following rules of operation. Γ and Δ are sets (multisets) and A and B are atomics.

Operational Rules

Negation

$$\frac{\Gamma \Rightarrow \Delta, A}{\neg A, \Gamma \Rightarrow \Delta} \neg L \qquad \frac{A, \Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, \neg A} \neg R$$

Conjunction

$$\frac{A, B, \Gamma \Rightarrow \Delta}{A \wedge B, \Gamma \Rightarrow \Delta} \wedge L \qquad \frac{\Gamma \Rightarrow \Delta, A \quad \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \wedge B} \wedge R$$

Disjunction

$$\frac{A, \Gamma \Rightarrow \Delta \quad B, \Gamma \Rightarrow \Delta}{A \vee B, \Gamma \Rightarrow \Delta} \vee L \qquad \frac{\Gamma \Rightarrow \Delta, A, B}{\Gamma \Rightarrow \Delta, A \vee B} \vee R$$

Implication

$$\frac{\Gamma \Rightarrow \Delta, A \quad B, \Pi \Rightarrow \Lambda}{A \rightarrow B, \Gamma, \Pi \Rightarrow \Delta, \Lambda} \rightarrow L \qquad \frac{A, \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \rightarrow B} \rightarrow R$$

Structural Rules

Weakening

$$\frac{\Gamma \Rightarrow \Delta}{A, \Gamma \Rightarrow \Delta} WL \qquad \frac{\Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, A} WR$$

Contraction

$$\frac{A, A, \Gamma \Rightarrow \Delta}{A, \Gamma \Rightarrow \Delta} CL \qquad \frac{\Gamma \Rightarrow \Delta, A, A}{\Gamma \Rightarrow \Delta, A} CR$$

Exchange

$$\frac{\Gamma, A, B, \Pi \Rightarrow \Delta}{\Gamma, B, A, \Pi \Rightarrow \Delta} XL \qquad \frac{\Gamma \Rightarrow \Delta, A, B, \Lambda}{\Gamma \Rightarrow \Delta, B, A, \Lambda} XR$$

File Layout

Sequent derivations are represented in code as a tree of **SequentNode** objects. Each node stores its antecedent and succedent formula lists, the rule that was applied (**SequentNodeRule** enum), and pointers to parent node(s). **UnarySequentNode** handles rules with a single premise; **BinarySequentNode** handles rules with two premises (e.g., $\wedge R$, $\vee L$, $\rightarrow L$). The **SequentConverter** walks this structure and produces the corresponding truth tree in Willow-compatible format.

For example, the derivation:

$$\frac{\frac{\frac{A \Rightarrow A}{B, A \Rightarrow A} WL} {A, B \Rightarrow A} XL \quad \frac{B \Rightarrow B}{A, B \Rightarrow B} WL}{A, B \Rightarrow A \wedge B} \wedge R$$

is constructed bottom-up as a **BinarySequentNode** (for $\wedge R$) whose left and right parents are **UnarySequentNode** chains applying the structural rules.

Truth Tree Rules (Propositional Calculus)

Truth trees systematically test the satisfiability of a set of formulas by decomposing compound statements into simpler components, branching when multiple possibilities must be explored.

Decomposition Rules

A decomposable statement is any statement that is not a literal (i.e., not an atomic or the negation of an atomic). A checkmark (✓) is placed next to a formula after it has been decomposed.

Non-Branching (Linear) Rules

These rules add formulas to the current branch without splitting:

$$\frac{P \wedge Q}{P \quad Q} \qquad \frac{\neg(P \vee Q)}{\neg P \quad \neg Q} \qquad \frac{\neg(P \rightarrow Q)}{P \quad \neg Q} \qquad \frac{\neg\neg P}{P}$$

In each case, *both* resulting formulas are added to the same branch, one after the other.

Branching Rules

These rules split the current branch into two sub-branches:

$$\frac{P \vee Q}{P \mid Q} \qquad \frac{P \rightarrow Q}{\neg P \mid Q} \qquad \frac{\neg(P \wedge Q)}{\neg P \mid \neg Q}$$

Biconditional Rules

$$\frac{P \leftrightarrow Q}{\begin{array}{c|c} P & \neg P \\ \hline Q & \neg Q \end{array}} \qquad \frac{\neg(P \leftrightarrow Q)}{\begin{array}{c|c} P & \neg P \\ \hline \neg Q & Q \end{array}}$$

The vertical bar (|) denotes a branch point.

Closure Rules

A branch is **closed** (×) if a formula and its negation both appear on that branch:

$$\frac{\varphi \quad \neg\varphi}{\times}$$

A branch is **open** (○) and **finished** if all decomposable statements have been decomposed and no closure condition is met.

Interpretation

- If all branches close: the root formulas are jointly unsatisfiable.
- If any branch is open and finished: the root formulas are jointly satisfiable.

Applications

- **Validity:** To test $A_1, \dots, A_n \vdash B$, write $A_1, \dots, A_n, \neg B$ at the root. All branches close $\Rightarrow$ valid.
- **Tautology:** Write $\neg\varphi$ at the root. All branches close $\Rightarrow$ tautology.
- **Equivalence:** $\varphi \equiv \psi$ iff both $\{\varphi, \neg\psi\}$ and $\{\neg\varphi, \psi\}$ are unsatisfiable.

File Layout

The truth tree is serialized into Willow-compatible JSON by the `TreeSerializer`. Each node is an object with the following fields:

- `id`: unique integer identifier
- `text`: the formula string (e.g., "`A`", "`(A  $\wedge$  B)`", " `$\times$` ", " `$\circ$` ")
- `children`: array of child node IDs
- `decomposition`: array of node IDs that justify this node
- `parent`: ID of the parent node (absent for root)
- `antecedent`: ID of the node this was decomposed from
- `premise`: boolean indicating whether this is a root-level premise

The complete output is wrapped as:

```
{
  "nodes": [ ... ],
  "options": {
    "requireAtomicContradiction": true,
    "requireAllBranchesTerminated": true,
    "lockedOptions": false
  }
}
```

Putting Them Together: Sequent Derivations and Truth Trees

The central insight is that **sequent calculus derivations are truth trees turned upside down**.

Axiom and Closure

The LK initial sequent $A \Rightarrow A$ corresponds to truth tree closure. By $\neg L$, we derive $\neg A, A \Rightarrow$ (i.e., $\neg A, A \Rightarrow \perp$). In truth trees, both A and $\neg A$ on the same branch triggers closure ($\times$).

$$A \Rightarrow A \quad \approx \quad \begin{array}{c} A \\ \neg A \\ \times \end{array}$$

Structural Rules

Structural rules in LK have no formal counterparts in truth trees, but are reflected informally:

- **Contraction** $\approx$ A formula can be reused on the same branch
- **Weakening** $\approx$ Redundant/unused formulas on a branch
- **Exchange** $\approx$ Decomposition order is irrelevant

Negation

$\neg$ rules in LK are reflected in truth trees when applied twice (double negation $\neg\neg P \rightsquigarrow P$). Single applications of $\neg L/\neg R$ correspond to the formula's position on the left (assumed true) versus right (assumed false) of $\Rightarrow$.

Operational Rule Correspondences

L-rules (preserving truth upward) trigger decomposition of the unsigned formula. R-rules (preserving falsity upward) trigger decomposition of the negated formula.

L-Rules $\rightarrow$ Decomposition of $A \circ B$

$$\begin{array}{ccc} \frac{A, B \Rightarrow}{A \wedge B \Rightarrow_{\wedge L}} & \approx & \begin{array}{c} A \wedge B \\ A \\ B \end{array} & \frac{A \Rightarrow \quad B \Rightarrow}{A \vee B \Rightarrow_{\vee L}} & \approx & \begin{array}{c} A \vee B \\ \swarrow \quad \searrow \\ A \quad B \end{array} \\ \\ & & \frac{\Rightarrow A \quad B \Rightarrow}{A \rightarrow B \Rightarrow_{\rightarrow L}} & \approx & \begin{array}{c} A \rightarrow B \\ \swarrow \quad \searrow \\ \neg A \quad B \end{array} \end{array}$$

R-Rules $\rightarrow$ Decomposition of $\neg(A \circ B)$

$$\begin{array}{ccc} \frac{\Rightarrow A \quad \Rightarrow B}{\Rightarrow A \wedge B_{\wedge R}} & \approx & \begin{array}{c} \neg(A \wedge B) \\ \swarrow \quad \searrow \\ \neg A \quad \neg B \end{array} & \frac{\Rightarrow A, B}{\Rightarrow A \vee B_{\vee R}} & \approx & \begin{array}{c} \neg(A \vee B) \\ \neg A \\ \neg B \end{array} \\ \\ & & \frac{A \Rightarrow B}{\Rightarrow A \rightarrow B_{\rightarrow R}} & \approx & \begin{array}{c} \neg(A \rightarrow B) \\ A \\ \neg B \end{array} \end{array}$$

Summary Table

Sequent Rule	Truth Tree Operation	Type
$\wedge L$	Decompose $A \wedge B$ (non-branching)	Linear
$\vee L$	Decompose $A \vee B$ (branching)	Branch
$\rightarrow L$	Decompose $A \rightarrow B$ (branching)	Branch
$\wedge R$	Decompose $\neg(A \wedge B)$ (branching)	Branch
$\vee R$	Decompose $\neg(A \vee B)$ (non-branching)	Linear
$\rightarrow R$	Decompose $\neg(A \rightarrow B)$ (non-branching)	Linear
$\neg L/\neg R$	Double negation (when applied twice)	Linear
Axiom $A \Rightarrow A$	Closure ($\times$)	Leaf

Note on $\wedge L$ and $\vee R$

For this project, we define $\wedge L$ and $\vee R$ as follows:

$$\frac{A, B, \Gamma \Rightarrow \Delta}{A \wedge B, \Gamma \Rightarrow \Delta} \wedge L \qquad \frac{\Gamma \Rightarrow \Delta, A, B}{\Gamma \Rightarrow \Delta, A \vee B} \vee R$$

These versions preserve both operands in the premise, making them reversible (the inference line is a biconditional). This avoids the “lossy” behavior of the standard formulation, where one operand is discarded when reading bottom-to-top.